



Smart Sparse Retrieval: Learning-Enhanced Information Retrieval using LLM-Based Query Expansion

Akhilesh Negi, 221AI008
Anush Revankar, 221AI009

Agenda



- Introduction
- Literature Survey
- Outcome of Literature Survey
 - Problem Statement and Objectives
 - Existing Methodology
- Dataset details
- Implemented Methodology with planned enhancements/novelty
- Work done by each team member
- Results and Analysis for each team member's work
- Conclusion and Future Work
- Individual contribution

Introduction



1. When you search for something online, a search engine has to look through millions of documents and find the ones most relevant to your query.
2. Traditional search systems like BM25 work by matching the exact words in your query to words in the documents — this is called **sparse retrieval** (only a few words match at a time).
3. The problem: if you type "heart attack symptoms" but a document says "myocardial infarction signs," BM25 misses it because the words don't overlap. This is called the **vocabulary mismatch problem**.
4. One solution is **query expansion** — automatically adding more relevant words to your original query before searching, so more relevant documents can be found.
5. Recent advances in Large Language Models (LLMs) — AI systems like GPT — allow us to generate entire passages that answer a query, and use those passages to expand the query.
6. This project studies how effectively LLM-generated passages can improve sparse retrieval when combined through a smart ranking strategy.

Literature Review



Author/year	Methodology	Advantages	Limitations
Exp4Fuse: A Rank Fusion Framework for Enhanced Sparse Retrieval using Large Language Model-based Query Expansion (2025, Lingyuan Liu, Mengxiang Zhang) (Base Paper)	Uses zero-shot LLM query expansion to create an augmented query alongside the original query. Both queries are independently retrieved using a sparse retriever and then combined using a modified Reciprocal Rank Fusion to produce the final ranking.	Improves sparse retrieval without retriever retraining, achieves state-of-the-art results on MS MARCO and low-resource datasets, and avoids complex prompt engineering or dense retrievers by leveraging fusion.	Requires LLM inference which adds latency and computational cost; effectiveness depends on quality of generated expansion; still a multi-stage pipeline increasing processing overhead.
Query2doc: Query Expansion with Large Language Models (2023, Wang <i>et al.</i>)	Generate <i>pseudo-documents</i> for each query via few-shot prompting of an LLM, then expand the original query with these documents.	Improves sparse and dense retrieval: BM25 gains +3–15% on MS-MARCO/TREC DL without fine-tuning, and also boosts state-of-art dense models. It harnesses the LLM's vast knowledge to disambiguate queries.	Relies on the quality of LLM-generated pseudo-docs; if the LLM hallucinates or outputs irrelevant content, expansion can hurt performance. It also incurs extra cost for LLM inference.
Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE) (2023, Gao <i>et al.</i>)	HyDE pivots queries through <i>hypothetical documents</i> : an LLM is prompted to generate a “fake” document for the query, which is then encoded by a dense encoder (e.g., Contriever) to retrieve similar real documents.	Outperforms unsupervised baselines: significantly beats Contriever on zero-shot benchmarks (various QA, search, non-English tasks). It effectively grounds LLM's knowledge by the encoder, filtering hallucinations.	The generated hypothetical document may contain hallucinated or irrelevant content, requiring a robust encoder to filter out noise. Relies on a strong LLM and an unsupervised encoder; quality hinges on both.

Outcome of Literature Review



1. Modern high-performance retrieval systems now follow a multi-stage pipeline: **initial retrieval** → **LLM expansion** → **refined retrieval** → **reranking**, instead of a single retrieval step.
2. Pure LLM expansion can introduce **hallucinations** and irrelevant terms, so using retrieved documents to guide the expansion (**retrieval-aware expansion**) makes the results more accurate.
3. The major challenge is balancing **effectiveness** and **efficiency**, since **LLM expansion improves accuracy** but **increases latency** and **computational cost**, motivating smarter and lightweight sparse retrieval methods.

Problem Statement

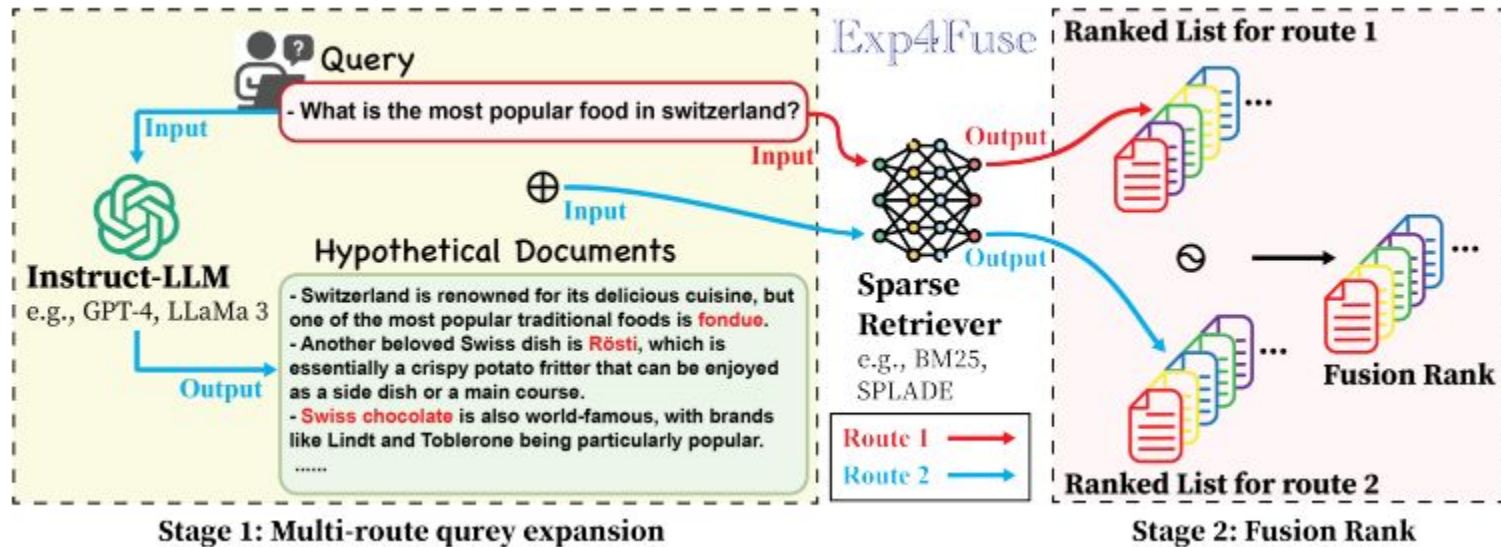


To develop a **lightweight, training-free** framework that integrates **zero-shot LLM-based query expansion** with **sparse retrieval**, addressing the **vocabulary mismatch problem** in information retrieval systems.

Research Objectives

1. Implement and study the **Exp4Fuse** framework — a dual-route sparse retrieval and rank fusion approach using zero-shot LLM-based query expansion.
2. Evaluate it on the **TREC DL 2019** benchmark using standard metrics (MAP, nDCG@10, Recall@1000).
3. Propose and experiment with **enhancements** to the base framework, including hyperparameter tuning and alternative fusion strategies.

Existing Methodology



Existing Methodology



1. The base paper proposes a two-stage framework called **Exp4Fuse** that uses a single search engine (BM25) but runs it twice per query.
2. **Stage 1 — Two retrieval routes:**
 - a. Route 1 (Original): The user's query is sent directly to BM25, which retrieves 1000 documents.
 - b. Route 2 (Expanded): The query is first sent to an LLM (like GPT-4), which generates a short hypothetical answer passage. This passage is then attached to several repetitions of the original query, and that combined text is sent to BM25 to retrieve another 1000 documents. Repeating the query multiple times (controlled by a value called λ , **lambda**) ensures BM25 still pays attention to the original query intent and doesn't get overwhelmed by the LLM-generated text.
3. **Stage 2 — Fusion Ranking:** The two lists of 1000 documents are merged using a formula called **Modified Reciprocal Rank Fusion (RRF)**. This formula gives each document a score based on its position (rank) in each list. Documents that appear in both lists get a bonus score, because appearing in both lists is a stronger signal of relevance.
4. The final merged and re-ranked list of top 1000 documents is the output.
5. The paper tested this on MS MARCO, TREC DL 2019/2020 (standard IR benchmarks), and 7 low-resource datasets (BEIR), showing consistent improvements over plain BM25.
6. Key default settings used in the paper: $\lambda = 5$ (query repeated 5 times), $k = 60$ (a smoothing constant in the fusion formula), equal weight for both retrieval routes.

Dataset Details / Statistics

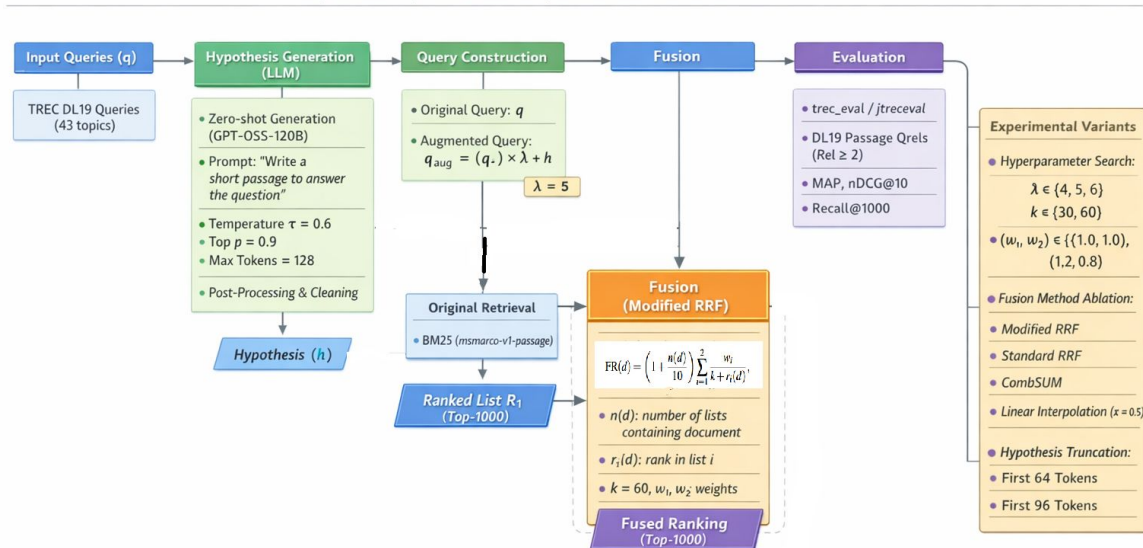


1. **TREC DL19 (Deep Learning 2019) Passage Task**
 - a. Passage corpus: ~8.8 million passages from MS MARCO.
 - b. Test set used in this project: **43 queries** with human relevance judgments.
 - c. Training data contains hundreds of thousands of queries (used in track development).
2. **MS MARCO Passage & Document Data**
 - a. Passage collection: ~8.8 million passages extracted from web documents.
 - b. Document collection: ~3.2 million documents derived from MS MARCO.
 - c. Used as the retrieval base for BM25 search in this project.
3. **Role in this Project**
 - a. TREC DL19 provides the **benchmark queries** and relevance judgments for evaluation.
 - b. MS MARCO provides the **large-scale corpus** over which BM25 retrieves passages/documents.

Implemented Methodology



Exp4Fuse Pipeline with Experimental Variants



Implemented Methodology



1. We first fully reimplemented the Exp4Fuse pipeline using the **Pyserini** toolkit (an open-source IR library) and the BM25 index for the MS MARCO passage collection, and verified our results match the base paper's reported numbers on TREC DL19.
2. For LLM hypothesis generation, we used **GPT-OSS-120B** via the **OpenRouter API** in **zero-shot mode** — meaning we gave the model no examples, just the question, and asked it to write a short plain-text passage answering it.
3. We sanitized (cleaned) the LLM output by stripping markdown symbols, table characters, and extra whitespace so that BM25 receives clean natural text.
4. On top of the baseline, we designed **three novelty experiments**, each probing a different underexplored aspect of the system.

Implemented Methodology



5. **Novelty 1 (Akhilesh):** We ran a **hyperparameter grid search** — systematically testing different values of λ (4, 5, 6), k (30, 60), and **route weights** (equal vs. slightly favouring the original query) to find the combination that works best. This produced **12 configurations** in total.
6. **Novelty 2 (Anush):** We ran a **fusion method ablation** — instead of only using the base paper's **Modified RRF**, we also tested **Standard RRF** (no bonus for appearing in both lists), **CombSUM** (add up normalized BM25 scores directly), and **Linear Interpolation** (a weighted average of normalized scores). This tells us whether the **rank-based merging approach** is actually the best choice.
7. **Novelty 3 (Anush):** We ran a **hypothesis truncation study** — we tested whether cutting the **LLM-generated passage** to only the **first 64 or 96 words** (instead of using the full ~100-word passage) changes retrieval quality. This is practically useful because shorter inputs are cheaper and faster.
8. All experiments were evaluated using three standard metrics: **MAP** (overall ranking quality), **nDCG@10** (quality of the top 10 results, Normalized Discounted Cumulative Gain), and **Recall@1000** (how many relevant documents appear anywhere in the top 1000).

Work done by Akhilesh Negi



1. Built the core **retrieval module** (`retrieval.py`) responsible for **loading BM25, constructing augmented queries, and writing output files in the standard TREC format** used by evaluation tools.
2. Built the **baseline replication script** that performs both BM25 runs (original and hypothesis-expanded) and the Modified RRF fusion, reproducing the base paper's pipeline end-to-end.
3. Built the **evaluation utility** (`eval_util.py`) that **automates running the official `trec_eval` scoring tool and parsing its output into structured result tables**.
4. Designed and executed **Novelty 1: Hyperparameter Grid Search** — ran all 12 configurations systematically by varying $\lambda \in \{4, 5, 6\}$, $k \in \{30, 60\}$, and weights $\in \{(1.0, 1.0), (1.2, 0.8)\}$, saving results to `novelty1_grid.csv`.
5. Analyzed the grid search results to identify that **$\lambda = 4$ is consistently better than $\lambda = 5$ or 6** , and that the RRF constant k has very little effect on performance in this two-route setup.
6. Wrote the Methodology, Novelty 1 experiment design, and results analysis sections of the project report.

Work done by Anush Revankar



1. Built the **LLM hypothesis generation pipeline** (`generate_dl19_hypothesis.py`) that connects to the OpenRouter API, sends each of the 43 TREC DL19 queries to GPT-OSS-120B, and saves the generated passages for reuse across all experiments.
2. Built the **unified experiment runner** (`run_experiments.py`) that ties all novelty experiments together with a command-line interface, saving all run files and result CSVs automatically.
3. Designed and executed **Novelty 2: Fusion Method Ablation** — compared **Modified RRF**, **Standard RRF**, **CombSUM**, and **Linear Interpolation** under identical retrieval conditions.
4. Designed and executed **Novelty 3: Hypothesis Truncation Study** — tested full, 96-token, and 64-token hypothesis lengths with all other settings fixed.
5. Wrote the Introduction, Related Work, Conclusion, and Novelty 2 and 3 analysis sections of the project report.

Results and Analysis



Akhilesh's Work (Baseline + Novelty 1)

TABLE I: Baseline Parity Results on TREC DL19 Passage Retrieval

System	MAP	nDCG@10	R@1000
BM25 (Original)	0.3013	0.5058	0.7501
BM25 (Hypothesis-Only)	0.3269	0.5475	0.7377
BM25 + Exp4Fuse	0.3208	0.5424	0.7756

Baseline Parity (confirming the system works):

1. **BM25 with LLM-expanded query only (no fusion):** The LLM expansion improves ranking quality but hurts recall (some relevant documents get pushed out) as compared to original BM25.
2. **Exp4Fuse (fusion of both):** Fusion recovers the recall loss while keeping most of the quality gain.
3. This confirms our implementation matches the expected behavior from the base paper.

Results and Analysis



Akhilesh's Work (Baseline + Novelty 1)

Novelty 1 — Hyperparameter Grid Search:

5. **Best configuration found:** $\lambda = 4$, $k = 30$, equal weights $\rightarrow$ $\text{nDCG@10} = \mathbf{0.5519}$, $\text{MAP} = \mathbf{0.3330}$, $\text{Recall} = \mathbf{0.7855}$ — all better than the default ($\lambda=5$, $k=60$).
6. **Reducing λ from 5 to 4 consistently improves all metrics** across all k and weight combinations — repeating the original query fewer times gives the LLM-generated passage a stronger relative voice in BM25's scoring.
7. **Increasing λ to 6 consistently degrades performance** — too many repetitions drown out the expansion terms entirely.
8. **The RRF constant k (30 vs. 60) makes almost no difference** — it is not a sensitive parameter in this two-route setup.
9. Asymmetric weights (1.2 for original, 0.8 for expansion route) give a slight nDCG@10 boost at $\lambda = 4$ (0.5535) but hurt at $\lambda = 5$, showing an interaction effect between these two settings.
10. **Key takeaway:** λ is the most important knob to tune; practitioners should reduce it below the paper's default of 5.

TABLE II: Novelty 1: Grid Search Results (selected configurations)

λ	k	w_1	w_2	MAP	nDCG@10	R@1000
4	30	1.0	1.0	0.3330	0.5519	0.7855
4	30	1.2	0.8	0.3283	0.5502	0.7853
4	60	1.0	1.0	0.3322	0.5526	0.7855
4	60	1.2	0.8	0.3272	0.5535	0.7848
5	30	1.0	1.0	0.3227	0.5427	0.7756
5	30	1.2	0.8	0.3193	0.5414	0.7784
5	60	1.0	1.0	0.3208	0.5424	0.7756
5	60	1.2	0.8	0.3183	0.5438	0.7784
6	30	1.0	1.0	0.3153	0.5389	0.7690
6	30	1.2	0.8	0.3126	0.5316	0.7717
6	60	1.0	1.0	0.3140	0.5364	0.7690
6	60	1.2	0.8	0.3112	0.5314	0.7727

Results and Analysis



Anush's Work (Novelty 2 + Novelty 3)

TABLE III: Novelty 2: Fusion Method Ablation on TREC DL19

Fusion Method	MAP	nDCG@10	R@1000
Modified RRF	0.3208	0.5424	0.7756
Standard RRF	0.3206	0.5424	0.7756
CombSUM	0.3309	0.5626	0.7765
Linear ($\alpha = 0.5$)	0.3309	0.5626	0.7765

Novelty 2 — Fusion Method Comparison:

1. **Standard RRF (no co-occurrence bonus):** Almost identical to Modified RRF, suggesting the *co-occurrence bonus* in *Exp4Fuse* adds no measurable value in this two-route setup.
2. **CombSUM (add up normalized BM25 scores):** Significantly better than both RRF variants.
3. **Linear Interpolation ($\alpha = 0.5$):** Same as CombSUM
4. Score-based methods outperform rank-based methods by **+0.020 nDCG@10** — a meaningful improvement. This works because both retrieval routes use the same BM25 model, so their raw scores are on the same scale and can be meaningfully compared after normalization.
5. **Key takeaway:** When both routes use the same retrieval model (homogeneous fusion), using actual scores rather than just positions gives better results. RRF's advantage is in mixing very different systems.

Results and Analysis



Anush's Work (Novelty 2 + Novelty 3)

TABLE IV: Novelty 3: Effect of Hypothesis Truncation on TREC DL19

Truncation	MAP	nDCG@10	R@1000
Full (~100 tokens)	0.3208	0.5424	0.7756
Truncated (96 tokens)	0.3208	0.5424	0.7756
Truncated (64 tokens)	0.3208	0.5424	0.7756

Novelty 3 — Hypothesis Truncation:

7. Full hypothesis (~100 tokens), Truncated to 96 tokens, Truncated to 64 tokens **give identical results**.
8. Cutting the hypothesis to 64 words **causes zero degradation in retrieval quality** — the most informative vocabulary appears in the first part of the LLM output anyway.
9. **Key takeaway:** You can use shorter (cheaper) LLM outputs without sacrificing retrieval quality, making the system more practical for real-world deployment.

Conclusion & Future Work



1. We **successfully replicated the Exp4Fuse framework on TREC DL19**, confirming that fusing two BM25 runs — one with the original query and one with an LLM-generated expansion — improves over plain BM25 on all three metrics.
2. From the **hyperparameter grid search**, we found that using $\lambda = 4$ (repeating the query 4 times instead of the default 5) consistently **improves performance**, and that the RRF constant k is not an important tuning parameter.
3. From the **fusion method ablation**, we found that **score-normalized methods (CombSUM, linear interpolation) outperform rank-based methods (RRF) by +0.020 nDCG@10** when both retrieval routes use the same underlying model, which is a significant and free performance improvement.
4. From the **truncation study**, we confirmed that **hypothesis length does not matter beyond a threshold** — 64 tokens is sufficient, meaning cheaper, shorter LLM outputs work equally well.
5. Future work can include applying these findings to **stronger learned sparse retrievers** like **SPLADE++** to check whether the same patterns hold beyond BM25.

Individual Contribution



Sl. No	Name of Team Member	Contributions
1	Anush Revankar	• The LLM hypothesis generation pipeline, • The fusion method ablation (Novelty 2), • The hypothesis truncation study (Novelty 3)
2	Akhilesh Negi	• Baseline replication script • The hyperparameter grid search experiments (Novelty 1), • The evaluation infrastructure (eval_util.py),.

References



1. Wang, X., Li, Y., Zhang, Z., et al., “**Query2Doc: Query Expansion with Large Language Models.**” *Proceedings of EMNLP*, 2023.
2. Gao, L., Ma, X., Lin, J., Callan, J., “**Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE).**” *Proceedings of ACL*, 2023.
3. Jagerman, R., Macdonald, C., Ounis, I., “**Query Expansion by Prompting Large Language Models.**” *Proceedings of SIGIR*, 2023.
4. Zhang, Y., Sun, H., et al., “**MuGI: Exploring Best Practices of Query Expansion with Large Language Models.**” *Proceedings of EMNLP Findings*, 2024.
5. Lei, W., He, B., et al., “**Corpus-Steered Query Expansion using Large Language Models.**” *Proceedings of ACL*, 2024.
6. Shen, T., Zhuang, H., et al., “**Retrieval-Augmented Retrieval: Large Language Model Enhanced Information Retrieval.**” *Proceedings of SIGIR*, 2024.
7. Liu, L., Zhang, M., “**Exp4Fuse: A Rank Fusion Framework for Enhanced Sparse Retrieval using Large Language Model-based Query Expansion.**” arXiv preprint arXiv:2506.04760, 2025.
8. Xia, P., Chen, Z., et al., “**Knowledge-Aware Query Expansion for Retrieval-Augmented Generation.**” *Proceedings of ACL*, 2025.